

Docker Practice Scenarios

Day 1: Containers, nginx and Ubuntu

Try each scenario in your own terminal. Run the commands yourself and note down any questions.

Scenario 1: The quick demo

Your teammate wants to see the default nginx page in a browser, and the container should clean up after itself.

1. Start an nginx container so it's reachable at `http://localhost:80`, with the terminal attached.
2. Stop it from a second terminal, using its container **ID**.
3. Why is there no leftover container afterwards?

Scenario 2: Running it in the background

Your terminal is now free, but you need nginx running as a background service named webhost.

1. Start nginx detached, with port 80 mapped and the name webhost.
2. Confirm it's running.
3. View its logs, then refresh the browser and view the logs again. What changed?
4. Stop it using its **name**, not its ID.

Scenario 3: The "why does rm fail?" puzzle

After Scenario 2, you run `docker container rm webhost` and get an error.

1. Explain why the error happens (hint: which flag did you use to start the container?).
2. Now start the same container **without** `--rm`, stop it, and run `docker container rm webhost`. What's different this time?
3. Try running `docker rm` on a container that is still running. What happens, and what's the correct order of commands?

Scenario 4: Peek inside the web server

You need to know where nginx serves its default page from, without installing anything on your machine.

1. Start nginx with an interactive shell instead of its default process, with port 80 mapped, the name webhost, and auto-removal.
2. Navigate to the folder that holds the website files and list them.
3. Exit the shell. What happens to the container, and why?

Scenario 5: Disposable Ubuntu vs persistent Ubuntu

You want a throwaway Linux box to test a package install.

1. Launch an interactive Ubuntu container named ubuntu that is removed on exit. Install `tree` inside it, then exit.
2. Run `docker ps -a`. Is the container there?

3. Launch another Ubuntu container named `ubuntu`, this time **without** `--rm`, then exit immediately.
4. Run `docker ps` and then `docker ps -a`. Why do the outputs differ?

Scenario 6: Resurrecting a stopped container

Continuing from Scenario 5, your `ubuntu` container is stopped but still exists.

1. Start it again without creating a new one.
2. Open a shell inside the **already running** container.
3. Check whether `tree` is installed. Why or why not?
4. Exit, then clean up completely, in the right order. Which command would fail if you ran it first?

Bonus Challenge: Port conflict

Start two `nginx` containers at the same time, both mapped to host port 80.

1. What error do you get?
2. Fix it so both run, then verify both in the browser.